SQL 인젝션 진단 및 대응

01 SQL 인젝션 개념과 발생 원리

02 SQL 인젝션 취약점 진단 및 대응

📌 학습 목표

- ✅ SQL 인젝션 공격의 개념과 원리를 이해한다.
- ✅ SQL 인젝션 공격의 다양한 기법을 학습하고 실습한다.
- ✅ SQL 인젝션을 방어하기 위한 보안 기법을 적용할 수 있다.

📍 학습 내용

1. SQL 인젝션 개념 및 발생 원리
2. SQL 인젝션 공격 기법 이해
3. SQL 인젝션 대응 기법

SQL 인젝션 취약점 진단 및 대응

01 SQL 인젝션 개념과 발생 원리

02 SQL 인젝션 취약점 진단 및 대응

SQL 인젝션 개요

개요

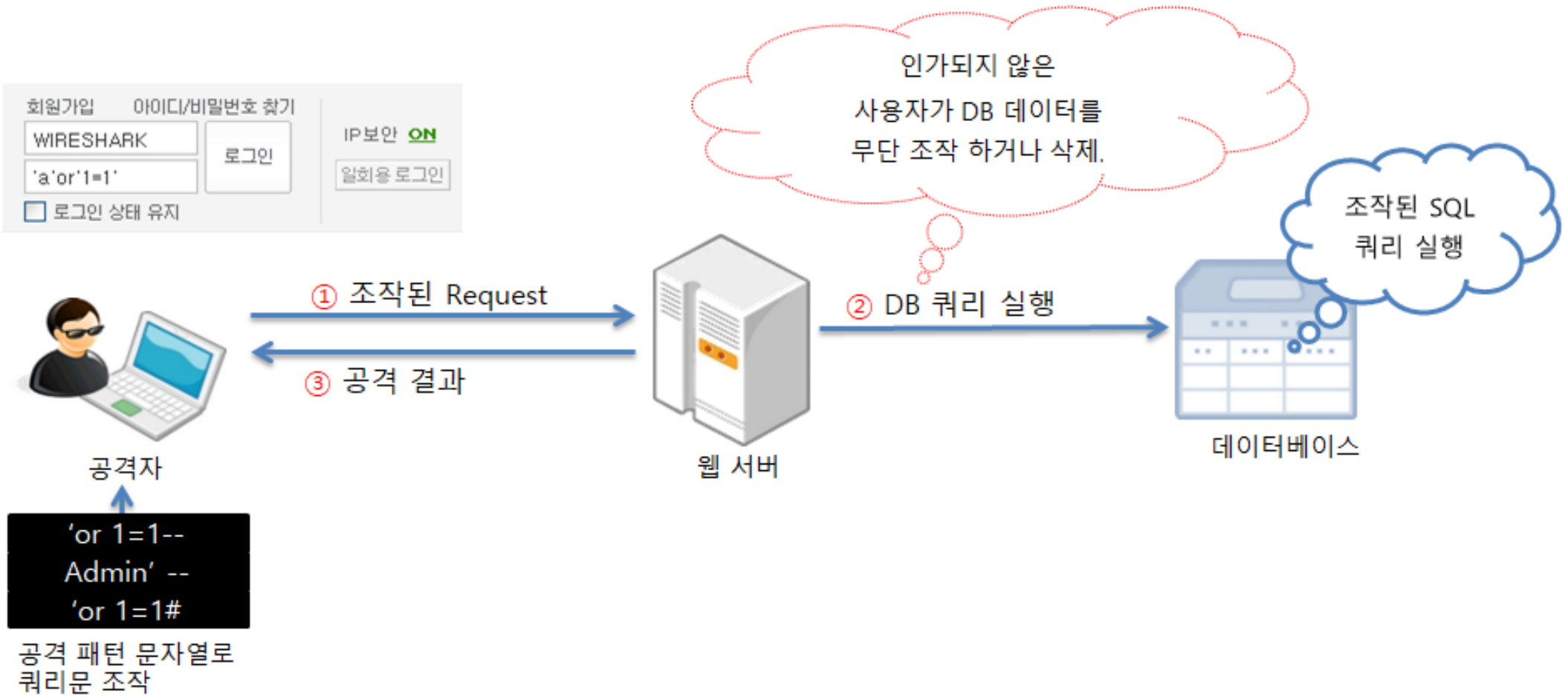
- 웹 애플리케이션에서 입력 받아 데이터베이스로 전달하는 정상적인 쿼리(CRUD)를 변조, 삽입하여 데이터베이스에 불법적인 데이터 열람, 시스템 명령 수행 등 비정상적인 데이터 베이스에 접근하는 기법.
- 모든 종류의 DBMS에 적용 가능한 공격 기법이며, 지속적으로 발전되고 있는 공격기법.

피해 내용

- 악성 스크립트 실행, 외부 프로그램 사용가능, DB 정보 열람,추가,수정,삭제 가능
- 프로시저를 통한 운영체제 명령어 수행, 불법 로그인 등의 침해 사고가 발생 가능



SQL 인젝션 개요


회원가입
아이디/비밀번호 찾기
WIRESHARK
'a'or'1=1'
로그인
IP보안 ON
일회용 로그인
로그인 상태 유지
인가되지 않은
사용자가 DB 데이터를
무단 조작 하거나 삭제.
조작된 SQL
쿼리 실행
① 조작된 Request
② DB 쿼리 실행
③ 공격 결과
공격자
웹 서버
데이터베이스
'or 1=1--
Admin' --
'or 1=1#
공격 패턴 문자열로
쿼리문 조작

SQL 인젝션 취약점 발생 원인

1. 동적 쿼리에서 사용자 입력 검증 부족

· 폼, URL 파라미터, API 요청 등 모든 외부 입력에 대해 검증 로직이 없는 경우 발생

· 입력 값 내 특수문자(' , ", ; 등)를 처리하지 않아 SQL 구문이 의도치 않게 변경됨

아래 SQL 문에서 userInput에 **admin' OR '1'='1** 을 삽입하면 항상 참이 되어 인증 우회됨

```
"SELECT * FROM users WHERE username = '" + userInput + "';"
```

SQL 인젝션 취약점 발생 원인

2. 에러 메시지 및 정보 노출

· 애플리케이션에서 내부 에러 메시지 및 데이터베이스 정보를 노출할 경우, 공격자가 이를 분석하여 추가 공격을 설계할 수 있음.

· 에러 메시지에 포함된 내용으로 다양한 정보 획득 가능

→ SQL 구문 오류, 테이블명, 컬럼 명, DB 버전 등

아래 SQL 문에서 userInput에 **`1' AND db_name() = 0--`** 삽입하면 에러 메시지에 데이터베이스 이름이 드러남.

```
"SELECT * FROM users WHERE username = '" + userInput + "';"
```

SQL 인젝션 취약점 발생 원인

3. 권한 및 보안 설정 미흡

· 애플리케이션에서 사용하는 DB 계정이 불필요한 권한(예: DROP, ALTER, EXECUTE 등)을 가지고 있을 경우, 공격자가 이를 악용할 수 있음.

· 취약한 저장 프로시저를 통해 시스템 명령어 실행이나 추가 권한 상승 공격이 가능.

SQL 인젝션 취약점 공격 방식

HTML

```
<form action="login.jsp">
    ID : <input type="text" name="id" />
    Password : <input type="password" name="password" />
</form>
```



Java

```
String sql = "select * from member where id = '" + request.getParameter["id"] + "' "
            + "and password = '" + request.getParameter["password"] + "' ";
statement.executeQuery[sql];
```

입력 예

정상 요청 : `login.jsp?id=gildong&password=gdhong`

공격 코드 : `login.jsp?id=a' or 'a'='a&password=a' or 'a'='a`

`login.jsp?id=admin' -- &password=x`

`login.jsp?id=admin' # &password=x`

Error-based SQL 인젝션 (오류 기반 SQL 인젝션)

추가 공격에 필요한 정보가 오류 메시지에 포함되어 노출될 수 있도록 입력 값을 조작

공격 방식 예시

- `'` ➔ 1개의 따옴표. 인젝션 가능 여부 및 에러 메시지 내용 확인이 가능
- `' having 1 = 1 --` ➔ 오류에서 테이블명, 컬럼 명 확인이 가능
- `' and db_name() = 1 --` ➔ 현재 데이터베이스의 이름 확인이 가능
- `' and 1 = (select @@version) --` ➔ 현재 설치된 SQL Server의 시스템 및 빌드 정보 확인이 가능

Union-based SQL 인젝션 (유니온 기반 SQL 인젝션)

조회 결과를 출력하는 사용자 화면에, 원래 화면 출력을 위한 쿼리와 공격자가 알고자 하는 정보를 조회하는 쿼리를 조인한 결과가 출력되도록 입력 값을 조작

공격 방식 예시

- `admin' union select 1,2,3,4,5,6 #`
- `admin' union select version(),2,3,4,5,6 #`
- `admin' union select schema_name,2,3,4,5,6 from information_schema.schemata #`
- `admin' union select group_concat(table_name),2,3,4,5,6 from information_schema.tables where table_schema=database() #`
- `admin' union select group_concat(column_name),2,3,4,5,6 from information_schema.columns where table_name='board_member' #`
- `admin' union select idx,userid,userpw,username,5,6 from board_member #`

Union-based SQL 인젝션 (유니온 기반 SQL 인젝션)

주요 DBMS별 시스템 테이블

DBMS	테이블	비고
MS-SQL	sysdatabases (sys.databases) sysobjects (sys.objects) syscolumns (sys.columns) systypes (sys.types) sysusers (sys.database_pricipals)	http://technet.microsoft.com/ko-kr/library/ms187997.aspx
Oracle	SYS.ALL_OBJECTS SYS.DBA_OBJECTS SYS.USER_OBJECTS SYS.TABLES SYS.ALL_TABLES SYS.DBA_TABLES SYS.USER_TABLES SYS.COLUMNS	http://docs.oracle.com/cd/E21901_01/doc/timesten.1122/e21644/systemtables.htm
MySQL	INFORMATION_SCHEMA.SCHEMATA INFORMATION_SCHEMA.TABLES INFORMATION_SCHEMA.COLUMNS	https://dev.mysql.com/doc/refman/5.7/en/information-schema.html

Blind SQL 인젝션 (블라인드 SQL 인젝션)

쿼리 실행 결과에 따라 서버의 반응이 다른 것을 이용한 공격

공격자가 알고자 하는 정보의 존재 여부를 확인하기 위해

참인 조건(쿼리 실행 결과가 존재)과 거짓인 조건(쿼리 실행 결과가 없음)을 번갈아 가면서 입력되도록 입력 값을 조작

Content-based 방식과 Time-based 방식이 있음

공격방식 예시

```
Data' and substring(user_name(1),1,1)='a' --      // a~z까지 첫번째 글자를 비교
Data' and substring(user_name(1),2,1)='a' --      // 응답 내용의 길이를 가지고 판단
```

또는

```
Data' and substring(user_name(1),1,1)='a' waitfor delay '0:0:5'     // 시간 지연을 가지고 판단.
```

. . . . 지속적으로 반복. 공격시간이 많이 걸리므로 툴에 의해 자동화 공격을 시도한다.

SQL 인젝션 공격 보안 사고 사례

데일리시큐

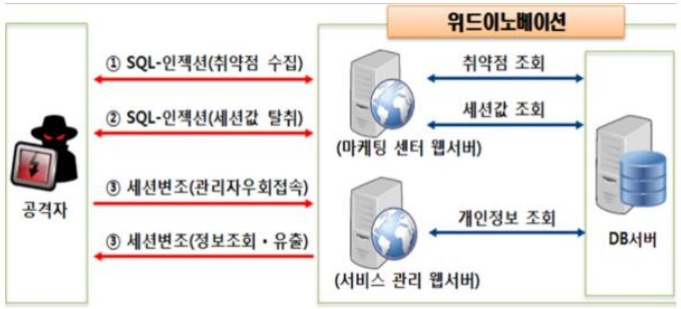
이슈 | 산업 | 정책 | 해외 | IT&생활 | 자료실 | 전체기사

최종편집 : 2025-02-16 19:45 (일)

여기어때, SQL 인젝션 공격 통해 회원정보 탈취 당해

길민권 기자 | 승인 2017.04.26 18:13

민관합동조사단, "홈페이지 취약점 공격으로 개인정보 유출 발생"



▲ 여기어때 해킹 과정 개요

방송통신위원회(이하 '방통위')와 미래창조과학부(이하 '미래부')는 지난 3월 7일(화)에서 17일(금)까지 발생한 위드이노베이션 '여기어때' 개인정보 유출 침해사고 관련 '민관합동조사단(이하 조사단)'의 조사 결과를 발표했다.

이번 조사는 위드이노베이션 서비스 이용고객을 대상으로 총 4,817건의 '협박성 음란문자(SMS)'가 발송됨에 따라 확인된 개인정보 유출 침해사고에 대한 신속한 대응과 사고 원인 분석을 통한 유사 피해 재발 방지 등을 위해 실시되었다.

시큐리티월드 보안뉴스

로그인 | 회원가입 | 기사제보 | 스크랩

통합검색

#전체기사 #시큐리티월드 #사건사고 #보안리포트 #사이버안보 #AI 보안

뉴스레터 신청 시큐리티 콘텐츠

Home > 전체기사

한국직업능력교육원, 해킹으로 개인정보 유출... 이름, 전화번호 등 4개 항목

입력 : 2024-09-10 11:39

한편 K테크에듀는 홈페이지 소개에 따르면 1999년 열린직업전문학교로 설립돼 25년의 역사를 이어오고 있다. 캠퍼스(교육원)는 시흥, 안산, 군포(안양), 인천 등 네 군데에 지점을 두고 있으며, 전체 교직원 수는 80여명에 이르는 것으로 파악된다.

이번 사고와 관련해 익스플로잇랩스 윤영 대표는 "결국 해킹으로 인한 개인정보 유출에 대한 완벽한 대응책이 없다는 것이 근본적인 문제"라며 "이번 사고를 살펴봤을 때 SQL 인젝션을 통해 사고가 발생한 것으로 보인다"고 말했다. 이어 "텔레그램에 유출된 데이터가 해킹을 통해 유출된 경우라면 상황이 심각하지만 일반 구글링을 통해 쉽게 얻을 수 있는 것들, 퍼블릭한 정보가 나간 것이면 이것도 보안이 취약한 것으로 봐야 하느냐에 대해서는 논란의 여지가 있다"며 "최근 기업들의 대응태세를 보면 일단 유출 사고가 났다고 하면 진위 여부부터 파악하고는 한다"고 설명했다.

SQL 인젝션 취약점 진단 및 대응

01 SQL 인젝션 개념과 발생 원리

02 SQL 인젝션 취약점 진단 및 대응

SQL인젝션 취약점 진단 방법 – 코드리뷰

코드 리뷰란?

- 개발자가 작성한 소스 코드를 체계적으로 검토하여 버그 및 보안 취약점을 찾아내는 프로세스
- SQL 인젝션과 같은 취약점은 코드 리뷰를 통해 조기에 발견할 수 있음

SQL 인젝션 취약점의 주요 원인

- 동적 SQL 사용: 사용자 입력 값을 문자열로 직접 쿼리에 삽입
- 입력 값 검증 미흡: 사용자 입력에 대한 필터링 또는 이스케이핑이 부족
- 잘못된 권한 설정: 데이터베이스 접근 권한이 과도하게 부여됨

코드 리뷰의 중요성

- 보안 취약점이 제품 출시 전 단계에서 사전에 제거됨
- 팀 내 보안 인식 제고 및 모범 사례 공유
- 자동화 도구와 수동 리뷰를 병행하여 취약점 탐지율 향상

SQL인젝션 취약점 진단 방법 – 코드리뷰

1. 동적 SQL 구문 사용 여부 점검

사용자 입력 값을 직접 쿼리 문자열에 결합하는 경우

```
public User findUserByUsername(String username) {
        String sql = "SELECT * FROM users WHERE username = '" + username + "'";
        try {
            return jdbcTemplate.queryForObject(sql, new BeanPropertyRowMapper<>(User.class));
        } catch (EmptyResultDataAccessException e) {
            return null;
        }
    }
```

2. 입력 값 검증 및 이스케이핑 점검

· 사용자 입력 값에 대해 화이트리스트 기반의 유효성 검증 수행

· 특수문자나 위험 문자에 대한 이스케이핑 적용 여부 확인

SQL인젝션 취약점 진단 방법 – 자동화된 진단 도구 활용

정적 분석 도구

소스 코드를 분석하여 보안 취약점(예: 동적 SQL, 입력값 처리 미흡 등)을 자동으로 탐지

예) SonarQube, Checkmarx, Fortify

동적 분석 도구

실행 중인 애플리케이션에 대해 실제 공격 시나리오(예: OWASP ZAP)를 적용해 취약점 검증

예) OWASP ZAP

자동화 도구의 필요성

· 광범위한 코드 검증: 수동 리뷰만으로는 놓치기 쉬운 취약점을 체계적으로 탐지

· 반복적 검사: CI/CD 파이프라인에 통합하여 코드 변경 시마다 자동 점검

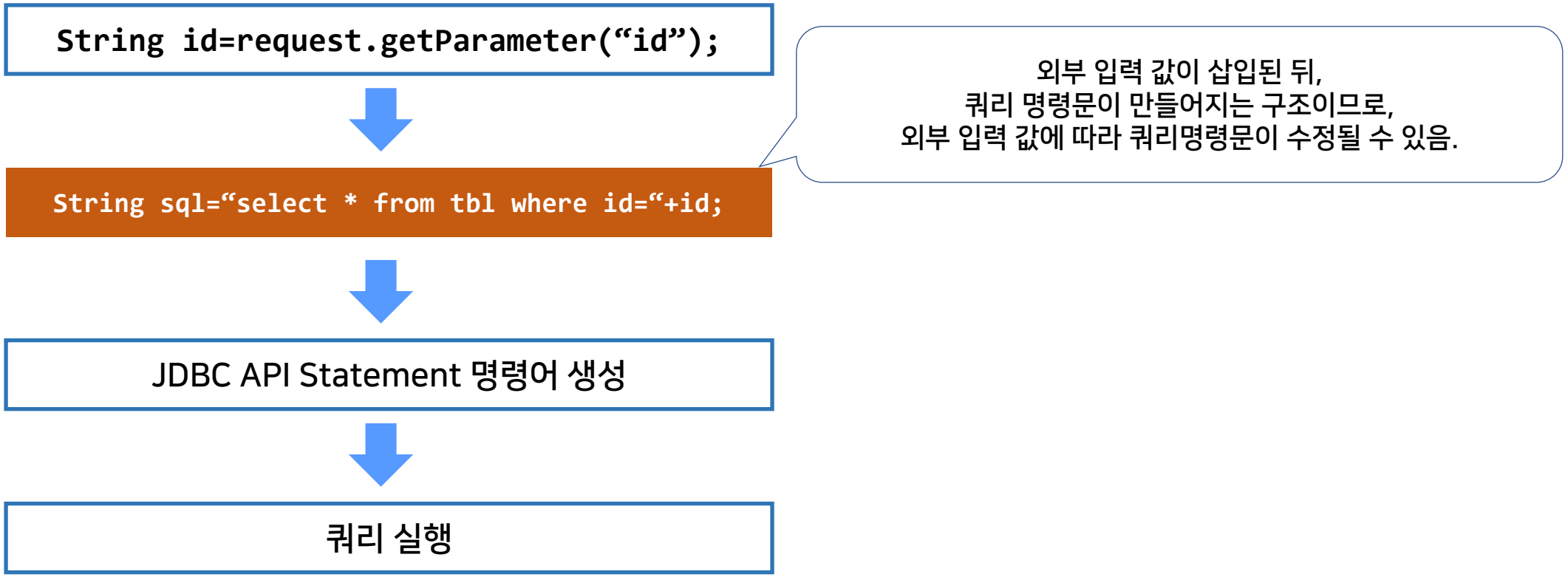
SQL 인젝션 취약점 탐지 역할

· 사용자 입력과 동적 SQL 구문 조합을 자동으로 스캔하여 위험 패턴을 식별

· 취약한 코드 패턴(예: 문자열 결합으로 SQL 생성)을 보고서로 제공

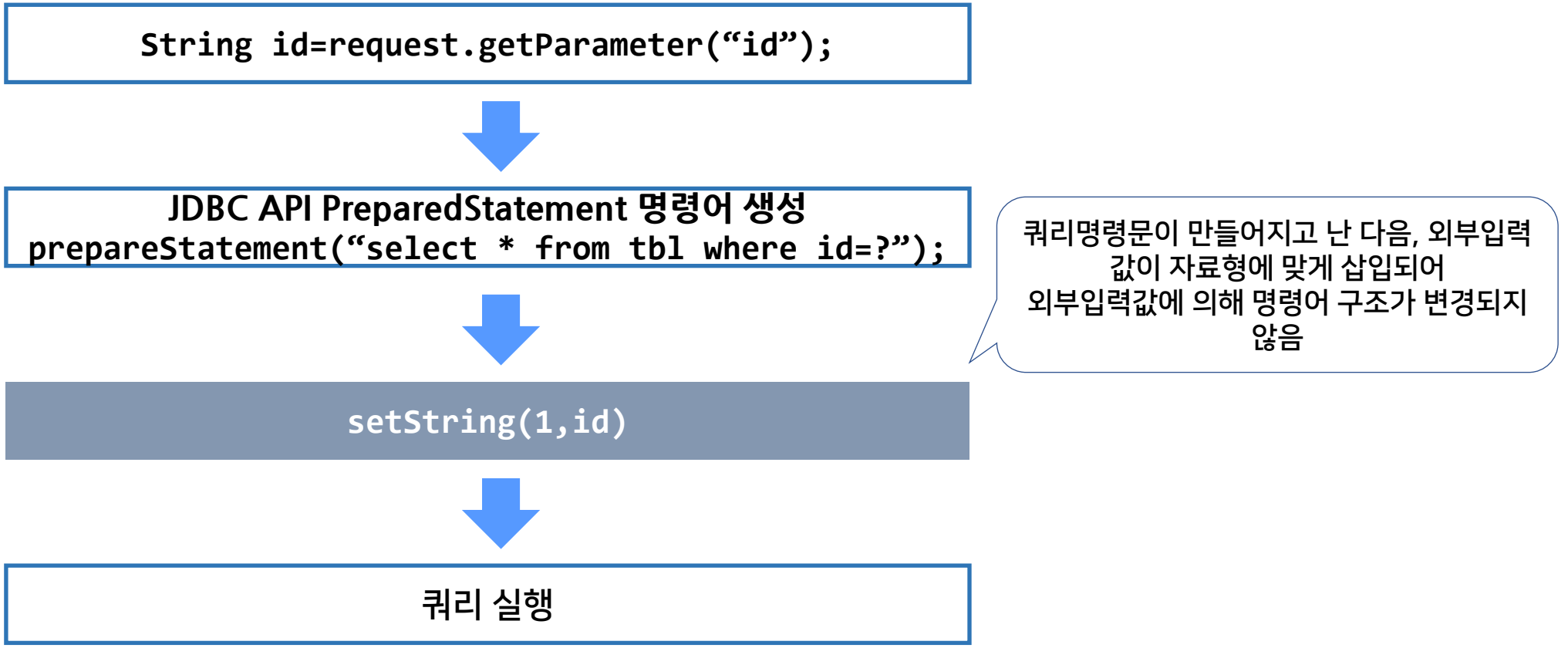
SQL인젝션 취약점 대응 방법 – 시큐어코딩 적용

취약성을 가지는 쿼리구조 - 동적 쿼리



SQL인젝션 취약점 대응 방법 – 시큐어코딩 적용

취약성을 가지지 않는 쿼리구조 - 동적 쿼리



SQL인젝션 취약점 대응 방법 – 시큐어코딩 적용

Statement 방식 DB Query

안전하지 않은 코드

```
String query = "SELECT * FROM users WHERE userid ='"+ userid + "'" +
                        “AND password='" + password + "'";
Statement stmt = connection.createStatement();
ResultSet rs = stmt.executeQuery(query);
```

안전한 코드

```
String query = "SELECT * FROM users WHERE userid=? AND password=?”;
PreparedStatement stmt = connection.prepareStatement(query);
stmt.setString(1, userid);
stmt.setString(2, password);
ResultSet rs = stmt.executeQuery();
```

PreparedStatement 의 setXXX()메소드로 외부 입력 데이터를 쿼리문에 설정하여 쿼리 구조가 변조되는 것을 방지한다.

SQL인젝션 취약점 대응 방법 – 시큐어코딩 적용



Hibernate Query Language 방식

안전하지 않은 코드

```
List results = session.createQuery(
               "from Orders as orders where orders.id = " + currentOrder.getId()).list();
```

안전한 코드

```
Query hqlQuery = session.createQuery("from Orders as orders where orders.id = ?0");
List results = hqlQuery.setString(0, "123-456-7890").list();
```

SQL인젝션 취약점 대응 방법 – 시큐어코딩 적용

MyBatis 방식

안전하지 않은 코드

```
<select id="getPerson" parameterType="String" resultType="org.application.vo.Person">
        SELECT * FROM PERSON WHERE NAME LIKE '${name}'
</select>
```

안전한 코드

```
<select id="getPerson" parameterType="String" resultType="org.application.vo.Person">
        SELECT * FROM PERSON WHERE NAME = #{name}
</select>
```

SQL인젝션 취약점 대응 방법 – 시큐어코딩 적용



Java Persistence API 방식

안전하지 않은 코드

```
List results = entityManager.createQuery
   ("Select order from Orders order where order.id = " + orderId).getResultList();
```

안전한 코드

```
Query jpqlQuery =
    entityManager.createQuery("Select order from Orders order where order.id= ?1");
List results = jpqlQuery.setParameter(1, "123-456-7890").getResultList();
```

SQL인젝션 취약점 대응 방법 – 시큐어코딩 적용

입력 값 필터링으로 안전한 값을 쿼리문에 사용

동적 쿼리 구조를 사용하여야 하는 경우, 외부 입력 값에 대해 철저한 필터링을 수행한 뒤 쿼리문에 사용하도록 한다.

전달되는 파라미터에 필터링 되어야 하는 특수 문자들

일반	URL Encoding	HTML Encoding
'	%27	' '
"	%22	" "
:	%3a	:
;	%3b	;
(	%28	(
)	%29	)
<	%3c	< <
>	%3e	> >
[	%5b	[
]	%5d	]
{	%7b	{
}	%7d	}

일반	URL Encoding	HTML Encoding
`	%60	`
=	%3d	=
#	%23	#
$	%24	$
%	%25	%
&	%26	& &
?	%3f	?
!	%21	!
@	%40	@
*	%2a	*
탭문자	%09	
|	%7c		

SQL인젝션 취약점 대응 방법 – 입력 값 검증

SQLMap을 활용한 검증 테스트

자동화된 SQL인젝션 테스트 도구 SQLMap을 활용하여, 구현된 입력값 검증이 제대로 작동하는지 테스트

검증 테스트 예시

```
sqlmap -u "http://example.com/search?keyword=test" --risk=3 --level=5 --batch
```

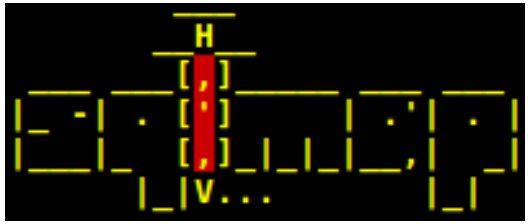
-u: 타겟 URL 지정 (GET 파라미터 포함)

--batch: 모든 질문에 자동으로 기본값 사용

--risk [1-3]: 위험 수준 설정 (높을수록 다양한 페이로드 시도)

--level [1-5]: 테스트 강도 조정 (높을수록 검사하는 파라미터와 페이로드가 많아짐)

--dump: 데이터베이스 내용을 덤프 (테스트 목적 시에만 사용, 운영 환경에서는 주의해야 함)



테스트 결과 판단

· "Parameter: keyword (GET) appears to be 'safe'" 출력 시 성공적으로 방어되었음

· SQL인젝션 관련 취약점 경고가 출력되면, 입력 값 검증 로직을 보완해야 함

SQL인젝션 취약점 대응 방법 – 보안 로그 기록 및 모니터링

로그 기록 및 모니터링의 필요성

조기 탐지

- SQL 인젝션 공격은 종종 비정상적인 쿼리 패턴이나 오류 로그를 남김
- 실시간 모니터링을 통해 공격 징후를 빠르게 파악 가능

사후 분석

- 공격 발생 시 로그를 통해 원인 분석 및 대응 전략 수립
- 법적 대응 및 보안 감사 자료로 활용 쿼리 파라미터: 숫자, 날짜, 식별자 등 쿼리에서 사용되는 모든 값

로그 기록 대상

- 웹 애플리케이션 요청 로그 (입력 값, URL, IP 등)
- 데이터베이스 에러 로그 (SQL 문법 오류, 타입 변환 오류 등)
- 인증/인가 관련 이벤트 (로그인 실패, 권한 상승 시도 등)

SQL인젝션 취약점 대응 방법 – 보안 로그 기록 및 모니터링

로그 기록시 고려사항

- 쿼리 파라미터 및 입력값
  사용자 입력값, 특히 특수문자나 SQL 예약어 포함 여부 기록
- 에러 및 예외 처리 로그
  SQL 실행 오류, 구문 오류, 타입 변환 오류 등 상세 에러 메시지 기록
- 사용자 및 요청 정보
  요청한 IP, timestamp, User Agent 등 공격 추적에 유용한 정보 기록

민감 정보(예: 비밀번호, 개인정보)는 로그에 기록하지 않도록 주의

모니터링 도구 및 기술

- SIEM(보안정보 및 이벤트 관리) 시스템
  Splunk, ELK Stack (Elasticsearch, Logstash, Kibana), Graylog 등을 활용하여 로그 집계 및 실시간 분석
- 알림 및 자동 대응
  - 특정 패턴(예: SQL 구문 오류, 이상한 파라미터 값) 감지 시 관리자에게 알림 발송
  - 일정 기준 초과 시 자동으로 방어 조치(예: IP 차단) 수행

요약 정리

- SQL 인젝션은 입력 값을 제대로 검증하지 않아 DB 쿼리가 조작될 수 있는 취약점입니다.
- 공격 방식에는 에러 메시지를 활용하는 Error-based, 여러 쿼리를 결합하는 Union-based, 응답을 통해 간접적으로 추론하는 Blind 방식 등이 있습니다.
- 이 취약점은 대규모 개인정보 유출이나 시스템 명령어 실행 등 심각한 보안 사고로 이어질 수 있습니다.
- PreparedStatement 같은 안전한 쿼리 작성, 입력 값 검증, 자동화 진단 도구 활용, 로깅·모니터링을 통해 대응하여야 합니다.
- 코드 리뷰와 사고 사례 학습을 통해 지속적으로 취약점을 점검하고, CI/CD 파이프라인에서 보안 검증을 강화해야 합니다.

참고 문헌

- [KISA] 소프트웨어 개발 보안 가이드(2021.12.29)
- [KISA] 소프트웨어 보안약점 진단가이드(2021)
- [오픈이지북스] 실무에 바로 적용하는 해킹 방어를 위한 JAVA 시큐어코딩
- [홍릉과학출판사] 소프트웨어 개발 보안 실무 - 시큐어 코딩 원리와 실제
- [secu BOOK] 웹 해킹과 시큐어코딩 탐지/수정 실습가이드
- [에이콘출판사] 보안 위협 모델링 Threat Modeling - 위협 식별과 대응을 위한 소프트웨어 설계
- [책만] 모두를 위한 소프트웨어 보안 설계와 구현 - 취약점 분석부터 위협 모델링과 완화 전략, 보안 패턴, 암호화까지 안전한 소프트웨어의 모든 것
- [위키북스] 웹 애플리케이션 보안 완벽 가이드 - 체계적으로 배우는 안전한 웹 애플리케이션 제작 기법
- [한빛미디어] 웹 애플리케이션 보안 - 정찰, 공격, 방어 세 단계로 배우는 웹 애플리케이션 보안의 모든 것
- [한빛미디어] 웹 해킹 & 보안 완벽 가이드 - 웹 애플리케이션 보안 취약점을 겨냥한 공격과 방어

감사합니다.



퀴즈

1. 다음 중 SQL 인젝션을 예방하기 위한 방법으로 옳지 않은 것은 무엇일까요?

1) PreparedStatement(또는 파라미터 바인딩)를 사용한다.

2) 사용자 입력값을 길이·형식·특수문자 등에 대해 검증한다.

3) 개발자가 만든 쿼리를 문자열 결합으로 직접 작성한다.

4) 데이터베이스 에러 메시지를 사용자에게 그대로 노출하지 않는다.

2. 다음 중 SQL 인젝션 공격으로 인해 발생할 수 있는 피해가 아닌 것은 무엇일까요?

1) 데이터베이스에서 민감 정보가 유출된다.

2) 관리자 계정으로 인증을 우회할 수 있다.

3) 서버의 파일이 삭제되거나 변경될 수 있다.

4) 네트워크 장비가 물리적으로 파손된다.

3. SQL 인젝션 방어를 위해 Java에서 동적 SQL 대신 권장되는 JDBC API 클래스 이름은?

( )



퀴즈 정답

1. **다음 중 SQL 인젝션을 예방하기 위한 방법으로 옳지 않은 것은 무엇일까요?**

   3) 개발자가 만든 쿼리를 문자열 결합으로 직접 작성한다.

   문자열 결합을 통해 쿼리를 직접 작성하면 인젝션 위험이 커집니다.

2. **다음 중 SQL 인젝션 공격으로 인해 발생할 수 있는 피해가 아닌 것은 무엇일까요?**

   4) 네트워크 장비가 물리적으로 파손된다.

   SQL 인젝션은 주로 DB 및 서버 내부 데이터 조작에 영향을 미치며, 물리적 장비 파손은 직접적인 영향 범위가 아닙니다.

3. **SQL 인젝션 방어를 위해 Java에서 동적 SQL 대신 권장되는 JDBC API 클래스 이름은?**

   (PreparedStatement)